

HW2: Twitter performance with Redis

DS 4300: Large-Scale Storage and Retrieval / Prof. Rachlin

DESCRIPTION

In this assignment we'll follow the lead of the Twitter engineers who recognized the scaling limits of a relational database by re-implementing our Twitter API to use Redis, a Key-Value store. You'll have some design choices to make in terms of what distinct keys you manage, and whether to put raw data into each value, or simply references to other keys in order to reduce storage overhead. As before, we are interested in the performance of two basic operations (measured in API calls per second):

postTweet: Post a tweet by a random user

getTimeline: Obtain the home timeline of a random user (i.e., for a random user, get the 10 latest tweets of all the users that your randomly chosen user follows.)

IMPLEMENTATION

If you structured your homework 1 correctly, you should not have to change very much in your main driver program which performs the timing analysis. You will be calling the same Twitter API methods, but now they are backed by a Redis implementation.

As you post each tweet, you copy the tweet (or a key name reference to the tweet) to each follower's home timeline automatically. Your write performance may now be slower, but since the timeline is ready and waiting, fetching the timeline should be a much faster operation. Let's find out how much faster!

As before, report the **postTweet** and **getTimeline** performance (API calls per second). Write up a summary report (1 page) and record your results on the Google Sheets document. Be sure to describe and document your storage model. Specifically, I want to know about the various categories of keys you are managing and what data structures they contain.

GRADING

Submit your code, including a documented Twitter API and its Redis implementation. You will be graded on the quality and completeness of your code and the design and implementation of your Redis-based Twitter API.